CineMatch - Movie Recommendation System

Complete Project Documentation

1. Project Overview

CineMatch is a modern, full-stack Movie Recommendation System. It uses Machine Learning (TF-IDF and Cosine Similarity) to recommend movies based on genre similarities. It features a highly interactive 3D frontend, dynamic search, multi-language filtering (English & Telugu), and automatic fallback data retrieval (OMDB API & Wikipedia API) for rich movie details.

2. Technology Stack

- **Frontend**: React, Vite, TailwindCSS, Framer Motion (Animations), Three.js / React Three Fiber (3D Elements).
- **Backend Framework**: Python, FastAPI, Uvicorn.
- **Machine Learning (Backend)**: Pandas, Scikit-learn (TF-IDF Vectorizer & Cosine Similarity).
- **Database (Backend)**: SQLite (`movies.db`).
- **APIs**: OMDB API (Primary movie details), Wikipedia REST API (Fallback data).
- **Deployment**: Vercel (Frontend), Render (Backend).

3. Backend Architecture (Python / FastAPI)

The backend serves as the "brain" of the application, handling data processing, machine learning logic, and serving endpoints to the frontend.

- `app.py`: The core server file. It loads the SQLite database on startup, vectorizes the movie genres using `scikit-learn`'s `TfidfVectorizer`, and computes a Cosine Similarity matrix to mathematically determine how related movies are to one another.
- **Endpoints**:
    - `/movies` (GET): Returns a list of all movies, with optional language filtering, used for the frontend search autocomplete.
    - `/recommend` (GET): Accepts a movie title, finds its index in the similarity matrix, and returns the top 6 highest-scoring recommendations.
- `requirements.txt`: Python dependencies including `fastapi`, `uvicorn`, `pandas`, `scikit-learn`, `numpy`, and `scipy`.
- `render.yaml`: CI/CD configuration to automatically deploy the FastAPI server to Render.com using a Web Service environment.

4. Frontend Architecture (React / Vite)

- `frontend/src/App.jsx`: The main application view. Handles user search input, communicates with the Python backend, filters by language, and triggers entrance animations for movie cards using Framer Motion.

- `frontend/src/components/Scene.jsx`: Creates the immersive 3D background. Uses Three.js floating octahedrons with a parallax effect tied to mouse movements.
- `frontend/src/components/MovieModal.jsx`: A cinematic popup that queries OMDB (and Wikipedia as a fallback) to retrieve and display movie posters, plots, ratings, and runtime dynamically.

5. Data Pipeline & Database

- `fetch_all_movies.py`: An automated data pipeline script that pulls the top movies from the past 15 years from TMDB (via proxy to bypass ISP blocks) and writes them to the SQLite database.
- `movies.db`: The resulting local SQLite database powering the system containing over 100 curated blockbuster movies across English and Telugu cinema.

6. Deployment URLs

- **Live Frontend URL**: https://movierecomended.vercel.app
- **Live Backend URL**: https://movie-recommendation-system-d6qr.onrender.com

When users interact with the Vercel site, API calls are securely routed to the Render backend to calculate recommendations, which are then enriched with IMDB/Wikipedia data directly in the browser.